



CEN207 Dönem Projesi

Proje Rehberi — 2026-2027 Güz

Dr. Öğr. Üyesi Uğur CORUH

Tek proje, iki kez

- Bu dönem **tek bir proje**: listeden seçtiğiniz bir uygulama
- Derste öğrenilen veri yapıları ve algoritmalarla **iki kez** gerçekleştirilir
- Önce **C** ile (vize kontrolü), sonra genişletilerek **Java** ile (final kontrolü)
- Amaç: "neden bu yapıyı seçtim?" sorusunu **sayılarla** yanıtlamak — karmaşıklık ve ölçüm, görüş değil

Kısaca — takım ve kontroller

- **Takım:** en çok 5 kişi (tek başınıza da olur)
- Bir konuyu **yalnız bir takım** alır; 3. haftadan sonra takım değişmez
- **Vize kontrolü (RAP1):** C gerçekleştirimi, rapor, gösterim — 7. hafta, 30.10.2026 — vizenin %60'ı
- **Final kontrolü (RAP2):** Java gerçekleştirimi, rapor, gösterim — 15. hafta, 25.12.2026 — finalin %70'i

Kısaca — araçlar, kapsam, son tarihler

- **Araçlar:** CMake + GoogleTest + Doxygen (C); JDK 21 + Maven + JUnit 5 (Java); Git ve GitHub
- **Kapsam:** her konuda bütün gereksinim kodları (V1–V6, F1–F9) zorunludur
- Konu kutusu, her kodun o uygulamada neyi temsil ettiğini söyler
- **Son tarihler:** konu/takım seçimi 3. hafta sonuna kadar (04.10.2026); plan onayı 4. haftada (09.10.2026)

1. Takvim

Proje takvimi

Aşama	Hafta	Tarih
Konu ve takım seçimi	3. hafta sonu	04.10.2026
Proje planı onayı	4. hafta	09.10.2026
Vize gösterimi + ara rapor (RAP1)	7. hafta	30.10.2026
Quiz-1	8. hafta	31.10–08.11.2026
Final gösterimi + final raporu (RAP2)	15. hafta	25.12.2026
Quiz-2	16. hafta	04–17.01.2027

2. Değerlendirme Yapısı

Proje not hesabını nasıl besliyor

$$Not_{Vize} = 0,6 \cdot RAP1 + 0,4 \cdot Quiz-1$$

$$Not_{Final} = 0,7 \cdot RAP2 + 0,3 \cdot Quiz-2$$

$$Baş\ arı\ notu = 0,4 \cdot Vize + 0,6 \cdot Final$$

Her kontrol hangi ÖÇ'leri ölçüyor

ÖÇ	Bloom düzeyi	RAP1	RAP2
ÖÇ.1 Doğrusal/doğrusal olmayan yapılar	Anlama	✓	✓
ÖÇ.2 Zaman/uzay karmaşıklığı (Büyük O)	Analiz	✓	✓
ÖÇ.3 Sıralama ve arama	Uygulama	✓	✓
ÖÇ.4 Ağaçlar ve hash tabloları	Uygulama	✓	✓
ÖÇ.5 Çizge algoritmaları	Uygulama	✓	✓
ÖÇ.6 Dosya organizasyonu	Değerlendirme	—	✓
ÖÇ.7 Yapı/algoritma seçimi	Sentez	✓	✓

3. Araçlar ve Kurulum

Araçlar — C tarafı (vize)

Araç	Ne için	Çıktı	Koşul
GCC/Clang/MSVC + CMake	C uygulamasını derleme	Çalıştırılabilir (teslim edilmez)	Hatasız, Windows + WSL/Linux
GoogleTest + gcov/lcov	Birim testi, kapsam	HTML kapsam raporu	%100 satır kapsamı
Doxygen	Kaynak kod belgesi	PDF	%100 kapsam; yalnız PDF, HTML klasörü yok

Araçlar — Java tarafı (final) ve ortak

Araç	Ne için	Çıktı	Koşul
JDK 21 + Maven	Java uygulamasını derleme	Sürüm (release) derlemesi	<code>mvn clean verify</code> , hatasız
JUnit 5 + JaCoCo	Birim testi, kapsam	HTML kapsam raporu	%100 satır kapsamı
Javadoc/Doxygen	Kaynak kod belgesi	PDF	%100 kapsam; yalnız PDF
Git + GitHub	Sürüm kontrolü	Özel (private) depo	Anlamli commit'ler, düzgün <code>.gitignore</code>
GitHub Actions (isteğe bağlı)	Sürekli entegrasyon	CI durumu	Etkinse, birleştirmeden önce yeşil olmalı

Şablondan depo oluşturun, adlandırın, paylaşın

- **Fork etmeyin:** herkese açık bir deponun fork'u özel yapılamaz
- Şablon sayfasında: **Use this template** → **Create a new repository**
- Sahibi seçin, ders koduyla adlandırın (kalıp bir sonraki slaytta), **Private**'ı seçin
- **Ders sorumlusunu** (ucoruh) ve **takım arkadaşınızı** collaborator ekleyin

Şablonlar ve depo adları

Kontrol	Şablon	Depo adı
Vize (C)	cpp-cmake-ctest-template	cen207-proje-ad-soyad-c
Final (Java)	eclipse-java-maven-template	cen207-proje-ad-soyad-java

Her iki şablon da derleme, birim testi, dokümantasyon üretimi, kapsam ölçümü ve paketlemeyi hazır verir — bunların **üzerine** inşa edersiniz.

Projenizi yerelde gösterme

GitHub Free'de özel (private) bir depoda GitHub Pages sitesi açılmaz. Her şeyi **kendi bilgisayarınızda** gösterirsiniz:

- `7-build-all-windows.bat` (Linux/WSL: `7-build-all-linux.sh`) derler, test eder, bütün raporları üretir
- `9-open-site-windows.bat` (Linux: `9-open-site-linux.sh`) tam siteyi `http://localhost` adresinde açar: bütün raporlar (testler, kod kapsamı, dokümantasyon kapsamı; Windows ve Linux) ve API belgeleri
- `release/` bütün çıktıları tutar: uygulama/exe, kütüphane, raporlar, API belgeleri, `site.zip`, kaynak kod, `ASSETS.md`, `SHA256SUMS.txt`
- `10-release-windows.bat` (Linux: `10-release-linux.sh`) GitHub Release'i oluşturur; sürümler özel depolarda da çalışır

Gösterim kontrol listesi

Gösterimde sırayla neyi açıp göstereceğiniz:

1. Yerel sitenin **ana sayfası** (`9-open-site-...` , `http://localhost` adresinde).
2. **Her rapor sayfası**: testler, kod kapsamı, dokümantasyon kapsamı (Windows ve Linux).
3. **API belgeleri**.
4. `release/` klasörünün içeriği.
5. Uygulamayı **release arşivinden çalıştırma**.
6. Özel deponuzun **GitHub Release sayfası** (ders sorumlusu collaborator olarak ekli).

lib / app / test düzeni

- **lib** — veri yapıları ve algoritmalar burada
- **app** — konsol menüleri ve kullanıcı etkileşimi; **lib** 'i kullanır
- **test** — birim testleri; **lib** 'i kullanır

Başlamadan önce: ortamınızın hazır olduğundan emin olun — **Ön Gereksinimler** sayfasına/sunumuna bakın.

4. Konu Seçimi

Konular nerede?

- Proje rehberi sayfasının sonundaki **Ek — Proje konuları listesi**
- 25'erli **sekiz grupta 200 konu**
- Her konu kutusu: kısa bir tanım, ardından her gereksinim kodu için tek cümle — o uygulamada **ne sakladığı ve hangi işlemi yaptığı**

Nasıl seçilir?

1. Listeyi inceleyin, bir konu seçin
2. Seçiminizi paylaşılan **takım ve konu listesine** yazın (bağlantı Teams duyurusunda) — bir konuyu bir takım alır, ilk yazan alır
3. **Proje planınızla** birlikte onaylatın — onaydan sonra konu değişmez

Konunun kendisiyle ilgili kurallar

- Konu kutusundaki eşlemeler **başlangıç önerisidir** — aynı gereksinimi daha doğal bir özellik karşılıyorsa raporda gerekçelendirerek değiştirin
- Listede olmayan bir fikir, bütün gereksinimleri anlamlı karşılıyorsa ders sorumlusunun onayıyla seçilebilir
- **Dersi tekrar alıyorsanız:** önceki projenizden farklı bir konu seçin

5. Gereksinimler

Kütüphaneden değil, sizden

Her kodun yanında **hangi haftada işlendiği** ve **ölçtüğü ÖÇ** vardır.
Her yapının temel işlemlerini (ekleme, silme, arama, listeleme...) **kendiniz** yazarsınız.

Hazır kütüphane yapıları (ör. Java `java.util`) bu gereksinimlerin yerine **sayılmaz**.

5.1 Vize kapsamı (C, 1–6. hafta) — V1–V3

Kod	Gereksinim	Hafta	ÖÇ
V1	Bağlı liste: çift + XOR/dairesel; ekleme, silme, arama, iki yönde gezinme	2	ÖÇ.1
V2	Seyrek matris: yalnız dolu hücreleri saklama; okuma, yazma, satır/sütun gezme	2	ÖÇ.1
V3	Yığın ve kuyruk: dizi ya da bağlı liste tabanlı	3	ÖÇ.1

5.1 Vize kapsamı (C, 1–6. hafta) — V4–V6

Kod	Gereksinim	Hafta	ÖÇ
V4	Ağaç ve öbek: ikili ağaç + 3 dolaşma; öbek tabanlı öncelik kuyruğu; öbek sıralaması	4	ÖÇ.1, ÖÇ.4
V5	Çizge ve dolaşma: komşuluk listesi/matrisi; BFS ve DFS	5	ÖÇ.1, ÖÇ.5
V6	Arama ve hash: ikili arama; hash tablosu + çakışma çözümü	6	ÖÇ.3, ÖÇ.4

5.2 Final kapsamı (Java, 9–14. hafta) — F1–F3

Kod	Gereksinim	Hafta	ÖÇ
F1	Java'ya taşıma: V1–V6'nın generics'li sürümü; tek menüde bütün özellikler	9–14	ÖÇ.1, ÖÇ.7
F2	Çizge algoritmaları: MST/en kısa yol/topo-sıralama/SCC/döngü/max-akıştan ≥ 2 'si	9	ÖÇ.5
F3	Sıralama: ≥ 3 algoritma, veri boyutlarında süreli karşılaştırma	10	ÖÇ.2, ÖÇ.3

5.2 Final kapsamı (Java, 9–14. hafta) — F4–F6

Kod	Gereksinim	Hafta	ÖÇ
F4	BST ve AVL: dengeleme dönüşleri; ekleme, silme, arama, aralık sorgusu	11	ÖÇ.4
F5	Dize algoritmaları: KMP/Boyer–Moore + düzenleme uzaklığı/LCS	12	ÖÇ.1, ÖÇ.3
F6	Trie ve ayrık kümeler: önek ağacı + union-find	9, 12	ÖÇ.4

5.2 Final kapsamı (Java, 9–14. hafta) — F7–F9

Kod	Gereksinim	Hafta	ÖÇ
F7	Dosya organizasyonu: sıralı + doğrudan erişimli dosya; dosyada çakışma çözümü	13	ÖÇ.6
F8	B+ ağacı dizini: dosya kayıtları için ikincil anahtar dizini	14	ÖÇ.4, ÖÇ.6
F9	Genişleyen dosyalar: genişletilebilir hash ya da dış birleştirmeli sıralama	14	ÖÇ.3, ÖÇ.6

5.3 Her iki kontrolde — uygulama

- **Konsol uygulaması, klavyeyle gezilen menü** — ok tuşları/Tab, yalnız numara girişi değil
- **Kalıcı veri binary dosyalarda** (`.bin` / `.dat`) — kayıtlar yeniden başlatmada kalır
- **Karmaşıklık:** her işlemin Büyük O'su, hem kod yorumunda hem raporda; en az iki yapı için ölçüm tablosu

5.3 Her iki kontrolde — mühendislik

- **Test ve belge:** birim test kapsamı **%100**, dokümantasyon kapsamı **%100**
- **Platform:** hem **Windows** hem **WSL/Linux**'ta derlenip çalışır
- **GitHub:** özel depo, anlamlı commit'ler, dal kullanımı, düzgün `.gitignore` ; derlenmiş dosya **yok**; Java deposu bir **sürüm (release)** üretir
- **Veri:** bütün veriler **sentetiktir** — gerçek kişi verisi asla kullanılmaz

6. Teslim Edilecekler

Kontrol başına ne teslim ediyorsunuz

Teslim	Vize	Final
Kaynak kod arşivi (derlenmiş dosya yok)	✓	✓
Rapor (.docx): tasarım, karmaşıklık, ölçüm, kapsam ekran görüntüleri	✓	✓ (güncellenmiş)
Doxygen/Javadoc çıktısı — yalnız PDF	✓	✓
Test kapsamı raporu (HTML, arşiv içinde)	✓	✓
Sunum (en çok 10 slayt)	—	✓
Video (kişi başı en çok 4 dk)	—	✓
Canlı gösterim ve sorular (~10 dk/takım)	7. hafta	15. hafta

Arşiv yapısı — vize (C)

```
cen207-vize-ad-soyad.zip
├── cen207-proje-ad-soyad-c/
│   ├── lib/          # V1-V6
│   ├── app/          # konsol menüleri, lib'i kullanır
│   ├── test/         # GoogleTest, lib'i kullanır
│   ├── CMakeLists.txt
│   ├── report/cen207-vize-ad-soyad.docx
│   ├── docs/cen207-vize-ad-soyad-doxygen.pdf
│   ├── coverage/     # gcov/lcov HTML
│   ├── .gitignore
│   └── README.md
```

Arşiv yapısı — final (Java)

```
cen207-final-ad-soyad.zip
├── cen207-proje-ad-soyad-java/
│   ├── src/main/java/...    # V1-V6'nın taşınmış F1, artı F2-F9
│   ├── src/test/java/...    # JUnit 5
│   ├── pom.xml
│   ├── report/, docs/, coverage/
│   ├── presentation/cen207-final-ad-soyad.pptx
│   ├── video/               # ziplemeden önce eklenir, commit edilmez
│   ├── .gitignore
│   └── README.md
```

Her şeyi adlandırma

Her dosyayı **ders kodu**, **kontrol adı** ve **ad-soyad** ile adlandırın:

`cen207-vize-ad-soyad.zip` · `cen207-final-ad-soyad.zip` ·
`cen207-vize-ad-soyad.docx`

Depo adları, araçlar/kurulum bölümündeki kalıbı izler.

7. Takım Çalışması ve Mühendislik Uygulamaları

GitHub Flow

- `main` korumalıdır — bütün çalışma **özellik dallarında** (`feature/hash-table`, `fix/avl-rotation`) yapılır
- Dallar **çekme isteğiyle (pull request)** birleştirilir, `main` 'e doğrudan push yok

Commit'ler ve inceleme

Conventional Commits: `feat(lib): implement AVL rotation` ·

`fix(app): correct menu navigation` · `test(hash): add collision unit tests`

Çekme istekleri: dalınızdan `main` 'e açın; takım arkadaşınız neyin değiştiğini ve nasıl test edildiğini inceleyip birleştirmeden önce onaylar

Issue'lar, CI, etiketler, Bitti Tanımı

- Her gereksinim kodu (V1–V6, F1–F9) için bir **GitHub Issue**; bir Projects panosunda izlenir (Backlog → Sürüyor → İncelemede → Bitti)
- **CI (etkinse)**: derleme/testler kırmızıyken asla birleştirmeyin
- **Sürüm etiketleri**: teslim ettiğiniz durum için `midterm-v1.0`, `final-v1.0`
- **Bitti tanımı**: uyarısız derleniyor, tam kapsamla test geçiyor, karmaşıklık belgelenmiş, takım arkadaşı kodu incelemiş

8. Rubrikler

Başarı düzeyleri (her kriter için)

Düzy	Anlamı
5 — Mükemmel	Her şey çalışıyor, testli, belgeli; gösterimde adım adım açıklanabiliyor
4 — İyi	Küçük eksik/uç durum hatası; genelde tam ve testli
3 — Yeterli	Temel işlemler çalışıyor; test/belge/ölçümde belirgin eksik
2 — Zayıf	Derleniyor ama çoğu işlem hatalı/eksik; açıklama zayıf
1 — Kanıt yok	Teslim edilmemiş ya da çalışmıyor

Puan = (düzy ÷ 5) × kriter puanı.

8.1 Vize rubriği — RAP1 (C, 100 puan)

#	Kriter	Kapsam	Puan
1	Doğrusal yapılar	V1 listeler, V2 seyrek matris, V3 yığın/kuyruk	20
2	Ağaç ve öbek	V4 dolaşmalar, öbek, öbek sıralaması	15
3	Çizge ve dolaşma	V5 gösterim, BFS/DFS	15
4	Arama ve hash	V6 ikili arama, hash, çakışmalar	10
5	Karmaşıklık analizi	Büyük O + ölçüm tablosu	10
6	Problem çözümüleme, yapı seçimi	Raporda "neden bu yapı?"	10
7	Yazılım mühendisliği	CMake, testler %100, belge %100, GitHub	20

8.2 Final rubriği — RAP2 (Java, 100 puan)

#	Kriter	Kapsam	Puan
1	Taşıma + tümleşik uygulama	F1 generics, tek menü, binary dosya	10
2	Çizge algoritmaları	F2, iki algoritma	10
3	Sıralama	F3, üç algoritma, karşılaştırma	10
4	Dengeli ağaçlar	F4 BST/AVL, dönüşler, aralık sorgusu	15
5	Dizeler ve yapılar	F5 arama/hizalama, F6 trie/union-find	15
6	Dosya organizasyonu	F7 dosyalar, F8 B+ dizini, F9 hash/sıralama	20
7	Mühendislik + sunum	Maven, testler %100, belge, video, gösterim	20

9. Kabul Koşulları

Teslim kabul edilmez, eğer...

- GitHub deposu yoksa, özel değilse ya da bir üyenin commit'i yoksa
- Birim test ya da dokümantasyon kapsamı **%100'ün altındaysa**
- Depoda/arşivde derlenmiş dosya varsa, Java deposunda sürüm yoksa
- Uygulama **Windows ya da WSL/Linux**'ta derlenip çalışmıyorsa
- **İntihal** tespit edilirse

10. Gösterimde Sorulacaklar

Gösterim soruları — Git/GitHub ve kurulum

- Depo şablondan doğru adla mı oluşturuldu? İki üyenin de commit'i var mı, dal kullanılmış mı?
- Birleştirme ve çakışmalar nasıl çözüldü? `.gitignore` doğru mu?
- **Windows ve WSL**'de derleyip çalıştırın; `lib` / `app` / `test` ve bağımlılıklarını gösterin

Gösterim soruları — veri yapıları ve testler

- Seçtiğiniz bir işlemi (ör. çift bağlı listede silme, AVL dönüşü, hash çakışması) **satır satır**, bellekte kutu-ok çizimiyle anlatın
- Karmaşıklığı nedir, neden?
- Test/dokümantasyon kapsamı raporlarını açın; bir uç durum testini gösterin

Gösterim soruları — dosya işlemleri ve programlama

- Kayıt ekleyin, programı kapatıp açın, binary dosyadan geri geldiğini gösterin
- **C:** işaretçiler/diziler, `struct`, `malloc` / `free`, dosya G/Ç, hata ayıklayıcıda çağrı yığını
- **Java:** generics, arayüzler, istisnalar

11. Mesleki Sorumluluk ve Dürüstlük

Etik, lisans, sentetik veri

- **Etik:** IEEE/ACM ilkeleri, kodunuzun ne yapıp yapmadığı konusunda dürüstlük ve adil teknik eleştiriye hem vermeyi hem kabul etmeyi ister
- **Lisans ve atıf:** raporda dış kaynakları belirtin; kullanılan kod parçasının kaynağını/lisansını yorumda yazın
- **Sentetik veri:** bütün veriler kendi ürettiğiniz örnek veridir — gerçek kişi verisi asla değil (bkz. kural 5.3)

İntihal ve ortak anlama

- Kod ve rapor **takımınıza** aittir
- Başka bir takımdan, önceki dönemden ya da internetten kopyalamak **intihaldir** — benzerlik denetimi yapılır, intihal **sıfır puan** demektir
- Her üye takımın **bütün** kodunu açıklayabilmelidir — gösterimde açıklanamayan kod **notlandırılmaz**

12. Sık Sorulan Sorular

SSS — takım ve konu

Takım yerine tek başıma çalışabilir miyim? Evet — en çok 5 kişi, tek başına çalışmaya izin var. Takımlar 3. hafta sonunda kesinleşir.

İki takım aynı konuyu isterse? Teams tablosuna ilk yazan alır.

Onaylı konuyu değiştirebilir miyim? Hayır — proje planıyla birlikte kesinleşir.

SSS — kapsam ve teslim

Kapsam %100'ün altında kalırsa? Kabul edilmez (bkz. kabul koşulları)
— son tarihten önce %100'e ulaşın.

Kütüphane koleksiyonları (ör. `java.util`) sayılır mı? Hayır —
temel işlemleri kendiniz yazarsınız.

Depoda/arşivde derlenmiş dosya olabilir mi? Hayır — `.gitignore` ile
kaldırın; varlığı reddedilme nedenidir.

Ek — Proje Konuları Listesi

Sekiz konu grubu

Aralık	Tema
001–025	Ulaşım, harita ve rota
026–050	Oyun ve bulmaca
051–075	Metin, dil ve arama
076–100	Bilim, sağlık ve biyoinformatik
101–125	Ağlar ve bilgisayar sistemleri
126–150	Lojistik, üretim ve ticaret
151–175	Medya, sosyal ağ ve kültür
176–200	Kent, çevre, afet ve tarım

Bir konu kutusu nasıl okunur

- Uygulamanın kısa bir **özet**i
- Ardından **her** gereksinim kodu için (V1–V6 ve F1–F9) tek cümle: o uygulamada **ne saklandığı ve hangi işlemin yapıldığı**
- Aynı gereksinim, her seferinde farklı bir hikâye

Örnek — 001 Metro Ağı Rota Planlayıcı (1/2)

Bir şehrin metro/tramvay/füniküler hatlarını tek ağda birleştiren; en hızlı ya da en az aktarmalı rotayı öneren konsol uygulaması (~12 hat, 250 istasyon).

- **V1** Her hat: istasyonların çift bağlı listesi; ring hatları dairesel liste
- **V2** Saat dilimi $\times$ istasyon yoğunluğu tablosu seyrek matris olarak saklanır
- **V3** Rota düzenlemeleri yığınla geri alınır; turnike girişleri kuyrukta
- **V4** Arıza ihbarları önem derecesine göre öbekte tutulur
- **V5** BFS en az duraklı yolu bulur; DFS kapanma sonrası kopan bölgeleri bulur
- **V6** İstasyon adı $\rightarrow$ kayıt hash tablosunda; sıralı kodlarda ikili arama

Örnek — 001 Metro Ağı Rota Planlayıcı (2/2)

- **F2** Dijkstra: dakika cinsinden en hızlı rota; Kruskal: en ucuz yeni hat ağı
- **F3** Rota seçenekleri süre/aktarma/mesafeye göre sıralanır, üç algoritma karşılaştırılır
- **F4** İstasyon başına sefer saatleri AVL ağacında — hızlı "X'ten sonraki ilk tren" sorgusu
- **F5** Yazılan istasyon adı KMP ile eşlenir; yazım hatası düzenleme uzaklığıyla düzeltilir
- **F6** İstasyon adları trie ile otomatik tamamlanır; union-find ulaşılabilir bölgeleri gruplar
- **F7–F9** Sefer kayıtları (sıralı dosya), istasyon kartları (doğrudan erişim + Brent yöntemi), kart numarasında B+ ağacı dizini

Diğer 199'unu nerede bulacaksınız

Ekin tamamı — sekiz grup, 200 konu kutusunun tümü — **proje rehberi sayfasında**, bu sunumun içeriğinin hemen ardından.

Göz atın, birini seçin, takım-konu tablosuna yazın.

Sorular?

Ders web sitesi: ucoruh.github.io/ce205-data-structures

Sırada: proje rehberi sayfasındaki tam konu listesine göz atın